



NTU Academy for Professional
and Continuing Education

(SCTP) Advanced
Professional Certificate

Data Science and AI





NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

3.4 Supervised Learning Advanced

Module Overview

3.1 Probability and Statistics for Machine Learning

3.2 Introduction to Machine Learning

3.3 Supervised Learning

3.4 Supervised Learning - Advanced

3.5 Unsupervised Learning

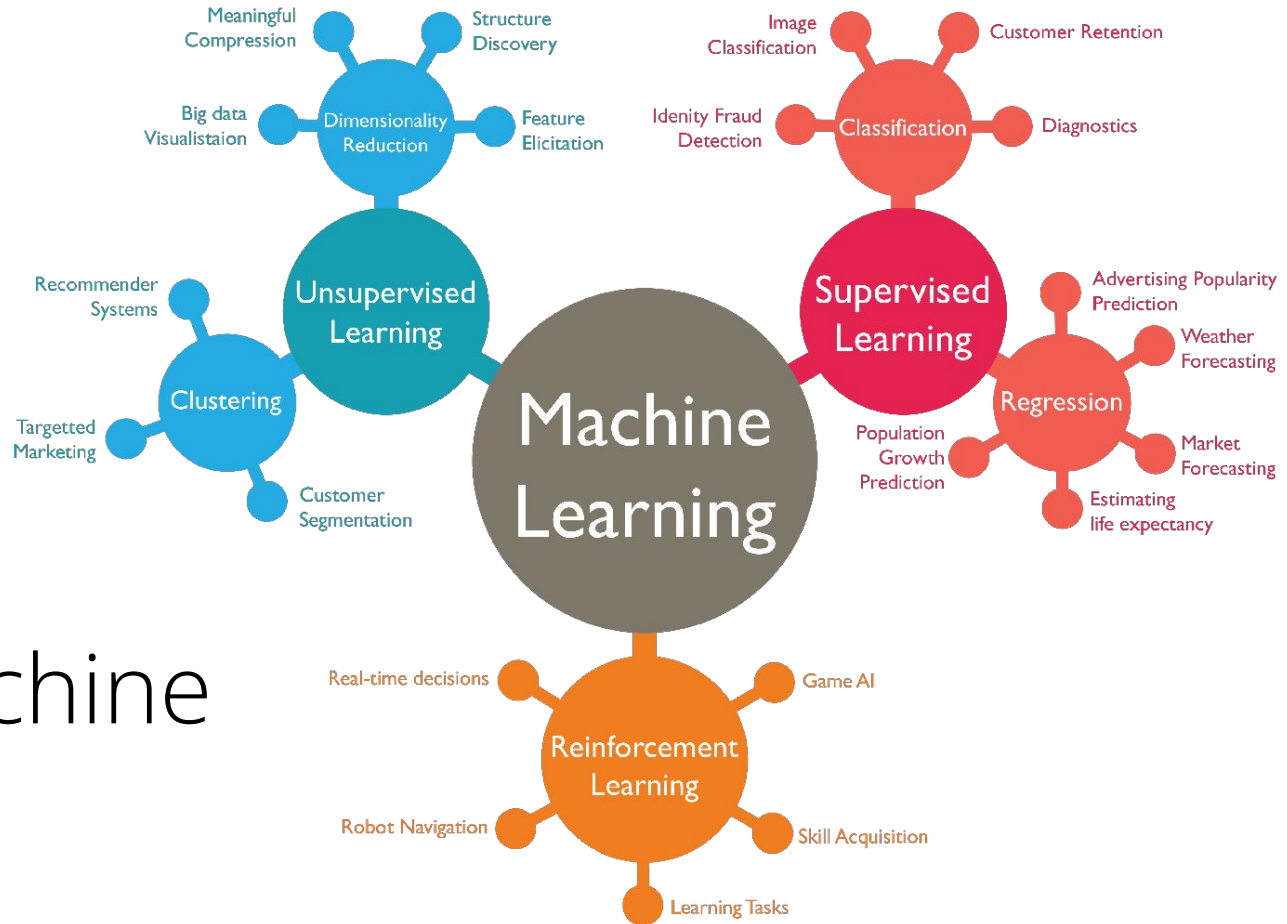
3.6 Time Series Data & Forecasting

3.7 Neural Network & Deep Learning

3.8 Computer Vision (CV)

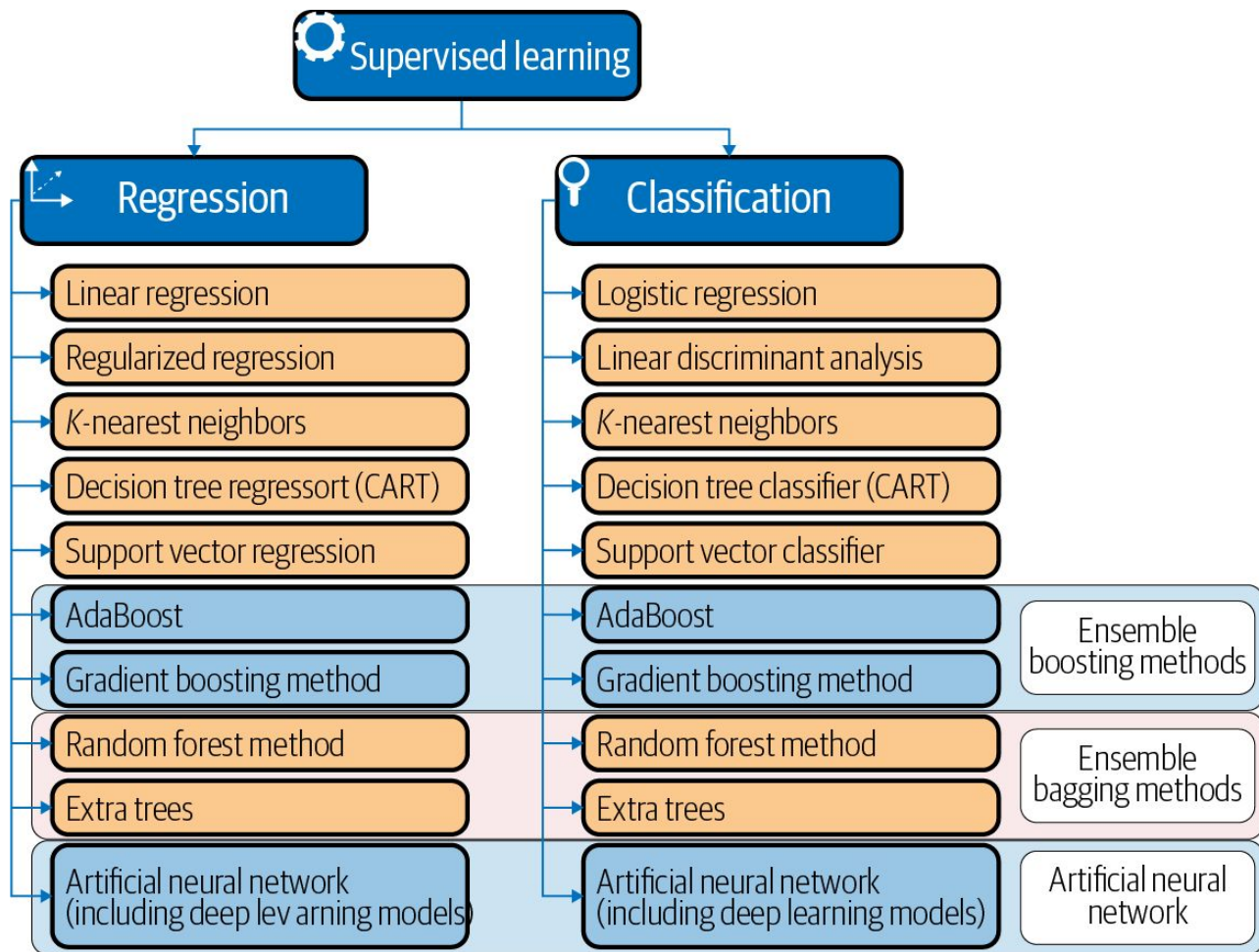
3.9 Natural Language Processing (NLP)

3.10 NLP - Advanced



Types of Machine Learning

Recap



Lesson Objectives

- Evaluate and tune hyperparameters
- Build end-to-end supervised learning workflow with advanced supervised learning models

Lesson Plan

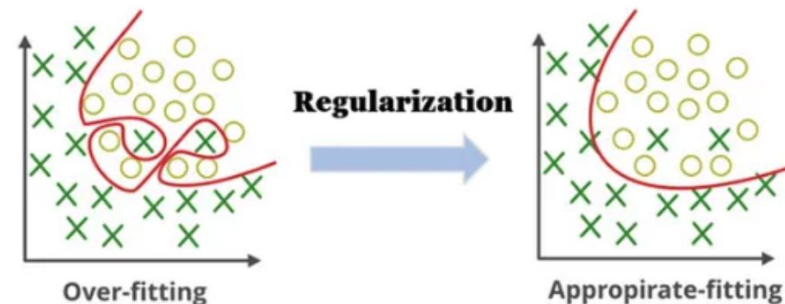
1. Regularization, Hyperparameters Tuning, Cross Validation and Grid Search
2. Model training workflow
3. Decision Trees
4. Bagging vs Boosting, Random Forest, Gradient Boosting, LightGBM

Regularization

What is Regularization?

Regularization is a technique used to prevent overfitting in machine learning models by penalizing complex models.

It involves adding a regularization term (penalty) to the [cost function](#) to discourage the model from learning overly complex patterns that just fit the training data.



Source:
<https://medium.com/@RobuRishabh/regularization-in-machine-learning-79e2f87ce898>

Types of Regularization

L1 (Lasso)

Adds the sum of the absolute values of the coefficients to the loss function.

It can lead to sparse models where some feature coefficients are zeroed out, effectively performing feature selection.

$$\text{Loss Function} = \text{MSE} + \alpha \sum_{i=1}^n |w_i|$$

L2 (Ridge)

L2 regularization adds the sum of the squares of the coefficients to the loss function.

It doesn't zero out coefficients but ensures they are small, leading to a less complex model.

$$\text{Loss Function} = \text{MSE} + \alpha \sum_{i=1}^n w_i^2$$

Types of Regularization

L1 (Lasso)

$$\text{Loss Function} = \text{MSE} + \alpha \sum_{i=1}^n |w_i|$$

L2 (Ridge)

$$\text{Loss Function} = \text{MSE} + \alpha \sum_{i=1}^n w_i^2$$

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$\text{Lossfunction} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \alpha \sum_{i=1}^n |w_i|$$

$$\text{Lossfunction} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \alpha \sum_{i=1}^n w_i^2$$

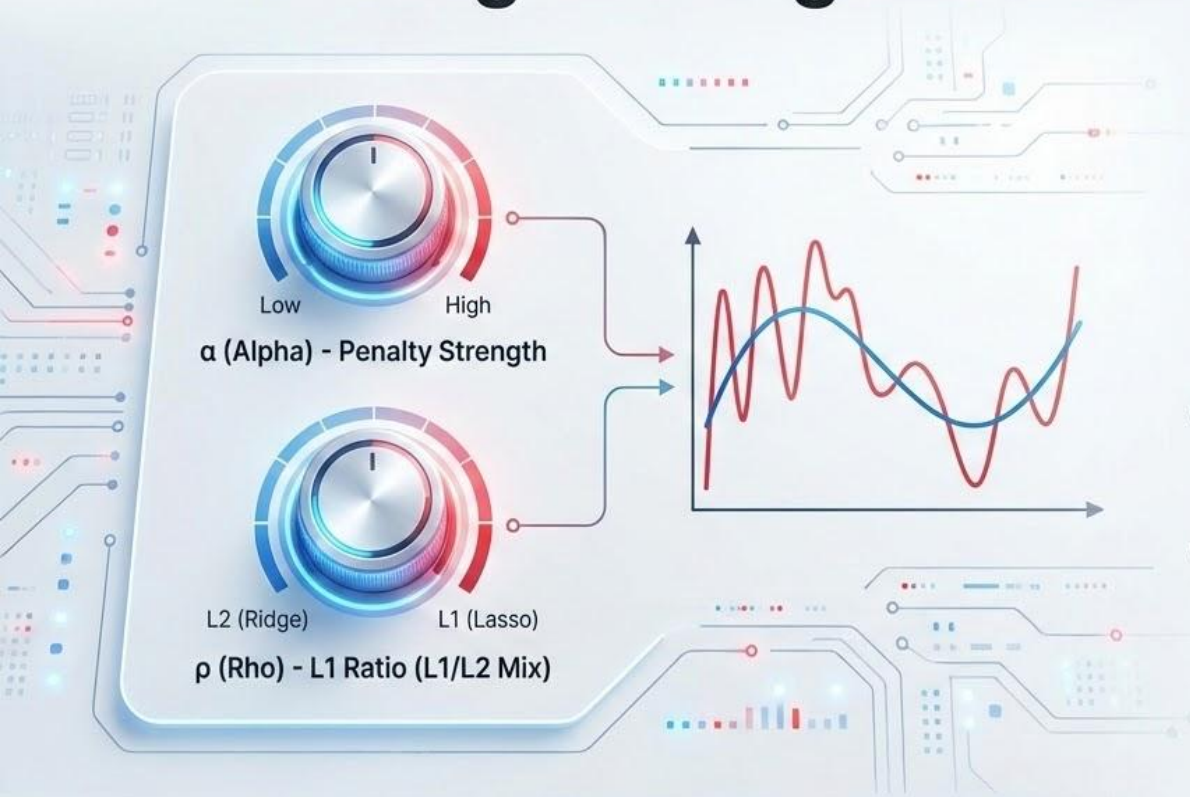
Elastic Net

Elastic Net is a combination of L1 and L2 regularization.

It is useful when there are multiple correlated features.

$$\text{Loss Function} = \text{MSE} + \alpha\rho \sum_{i=1}^n |w_i| + \alpha(1 - \rho) \sum_{i=1}^n w_i^2$$

Choosing the Regularization Parameter



Benefits of Regularization



Reduces overfitting by penalizing large coefficients.



Can lead to simpler models that generalize better.



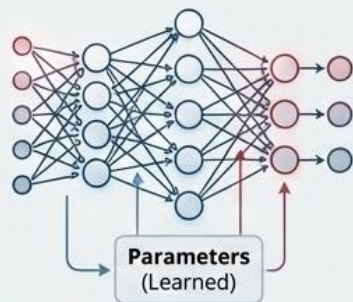
In the case of L1 regularization, can perform feature selection by zeroing out coefficients.

Hyperparameter Tuning, Cross-Validation and Grid Search

Hyperparameters

Parameters vs. Hyperparameters

In machine learning, models have parameters that are learned from the data and hyperparameters that are set by the practitioner.



The regularization parameter α and L1 ratio ρ are examples of hyperparameters.



Hyperparameter Tuning Process

Hyperparameter tuning is the process of finding the optimal combination of hyperparameters that yields the best performance.

Cross-validation and grid search are two techniques used for this purpose.

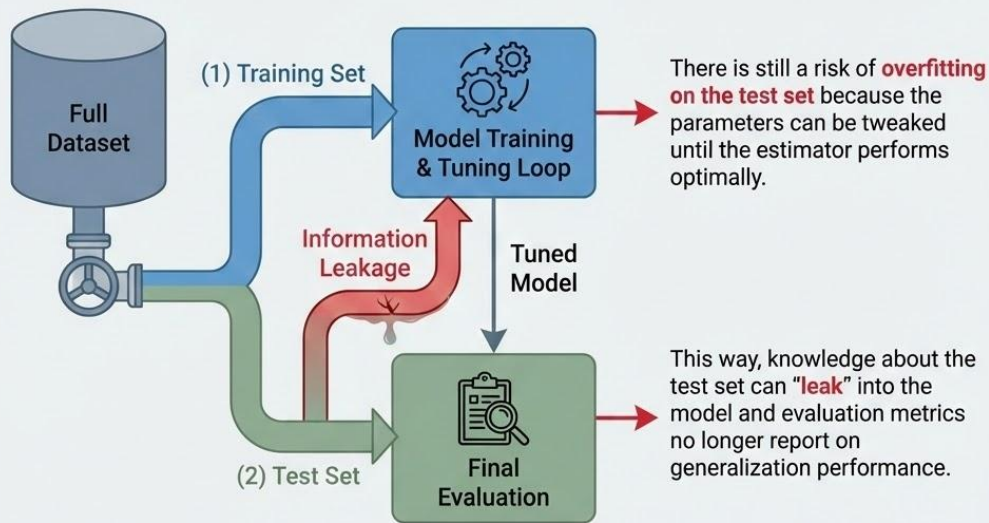


Cross-Validation

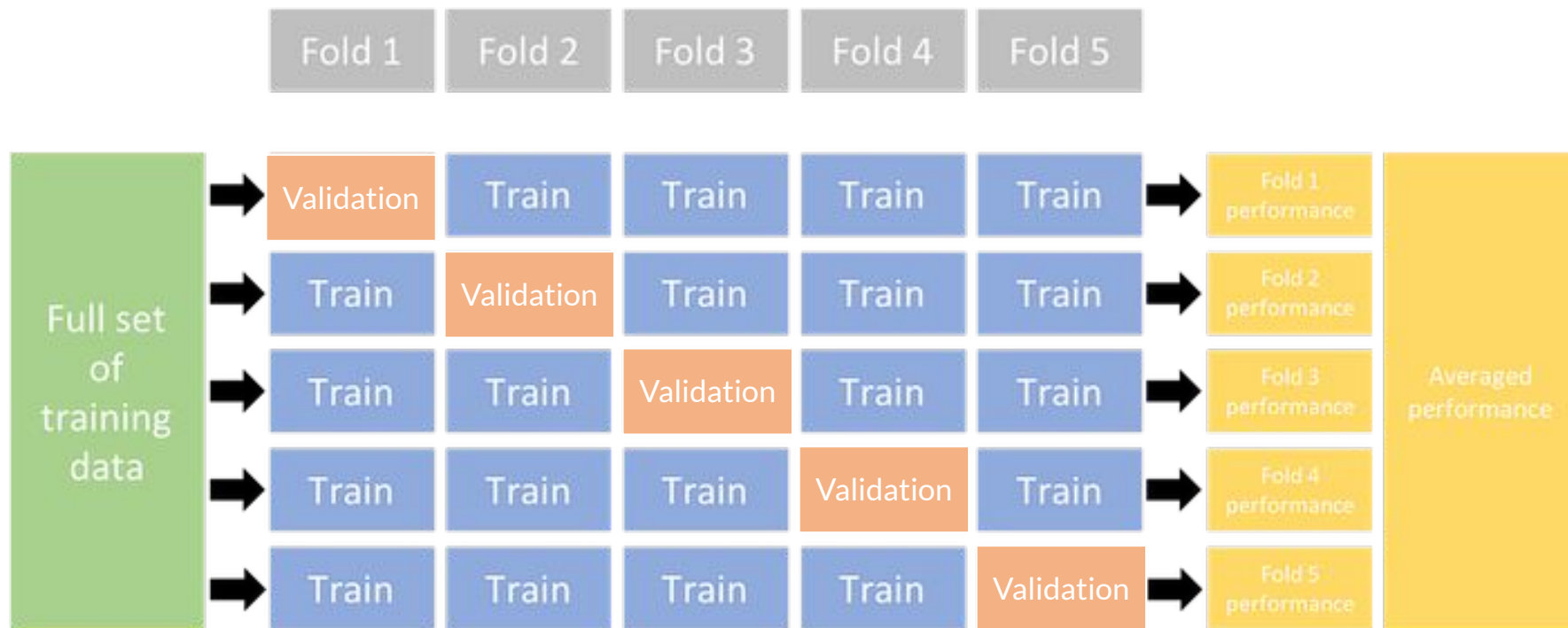
The train-test split is a simple method to divide your dataset into two parts: (1) for training the model and (2) for testing it.

However it has the following **disadvantage** if we use it for hyperparameters tuning:

Leaky Pipeline



Cross-Validation - K-Fold

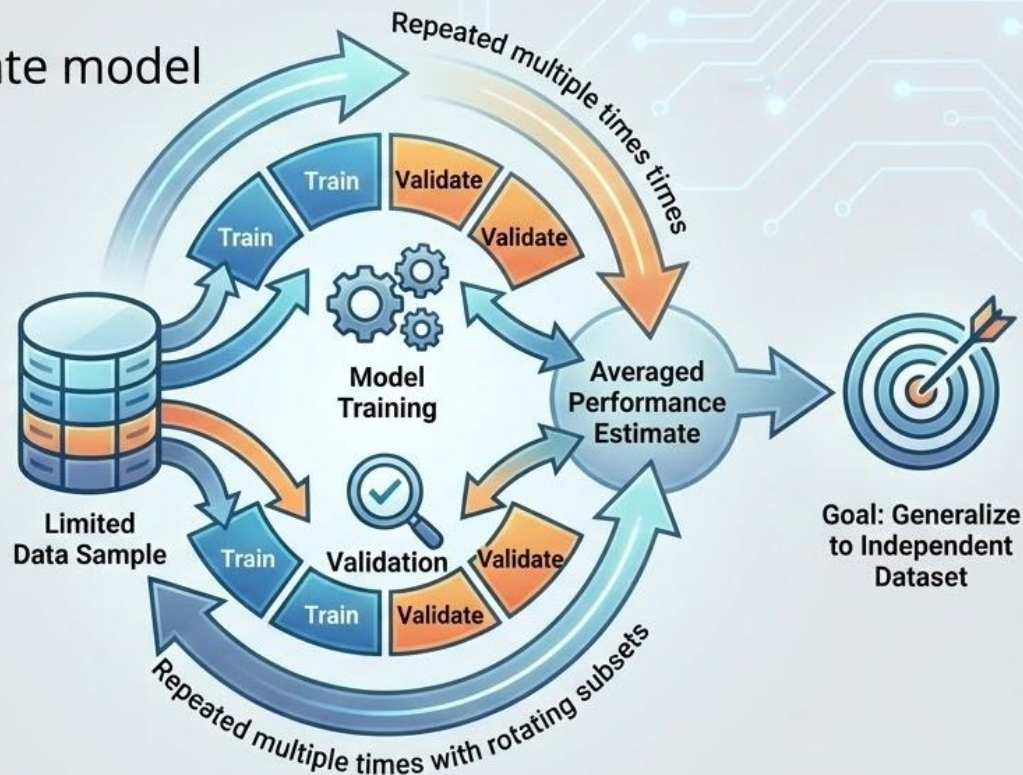


Cross-Validation

A resampling procedure to evaluate model performance on limited data.

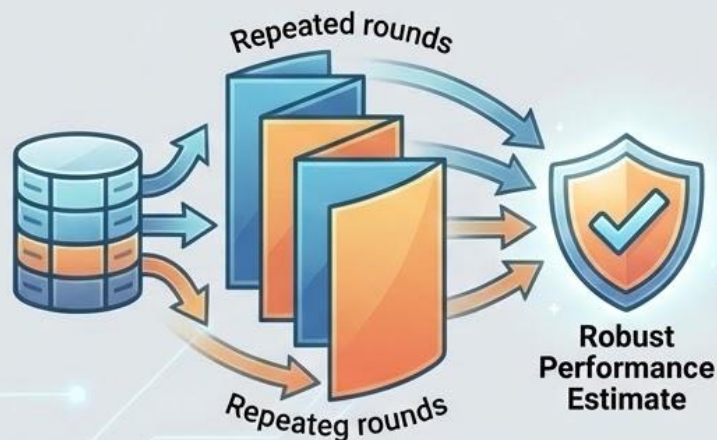
1. Partition data into subsets
2. Train model on some subsets
3. Validate on remaining subsets

Ensures robust assessment of how predictions generalize, similar to train-test split.



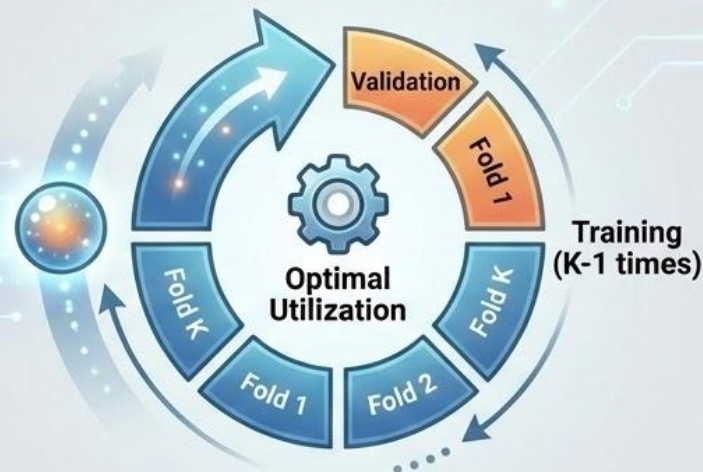
Cross-Validation

Reliable Estimate



More reliable estimate of out-of-sample performance due to multiple rounds of splitting.

Maximized Data Usage

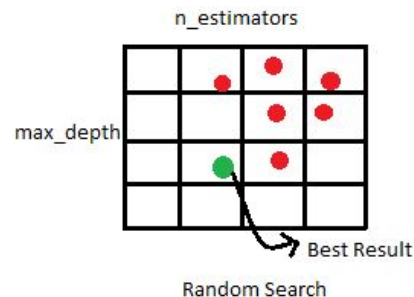
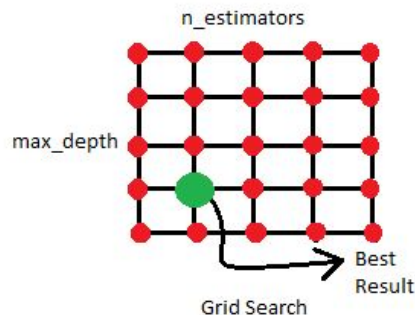


Makes better use of data as each data point gets to be in a validation set exactly once and in a training set **K-1** times.

Grid Search

Grid search is a brute force method to estimate the best hyperparameters. A grid of hyperparameter combinations is created, and the model is evaluated for each combination using cross-validation.

1. Define the hyperparameter grid.
2. Use cross-validation to evaluate each combination of hyperparameters.
3. Select the combination that yields the best performance.

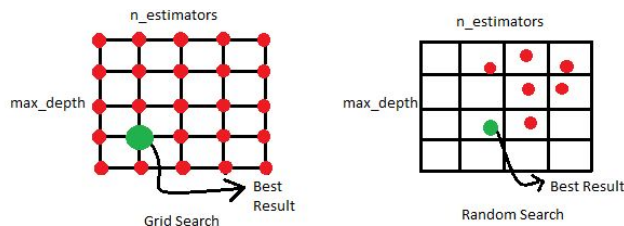


Random Search

Random search is an alternative to grid search. Instead of trying out every possible combination, it samples a random subset of parameter combinations.

- Can be faster than grid search when dealing with a large hyperparameter space.
- Can sometimes find a better combination of hyperparameters by exploring a wider range of values.

1. Define a search space as a bounded domain of hyperparameter values.
2. Randomly sample combinations of hyperparameters from this domain.
3. Perform cross-validation for each combination.
4. Select the combination that yields the best validation score.



Model Training Workflow

Step 1

After preprocessing of the data, train-test split is performed.

The test set is kept for the final evaluation of the tuned model.

Step 2

A combination of hyperparameters' prospective values is determined.

Then grid/random search is performed to determine the best possible combination candidate by using cross-validation to evaluate each candidate.

Step 3

The whole training set is used to train the model with the best candidate combination of hyperparameters found from the previous step.

This model is called the hyperparameter-tuned model.

Step 4

Evaluate the tuned model on the test set.

Code-along



Jupyter Notebook

Part 1: Regularization and Hyper-parameter Tuning

Code-along

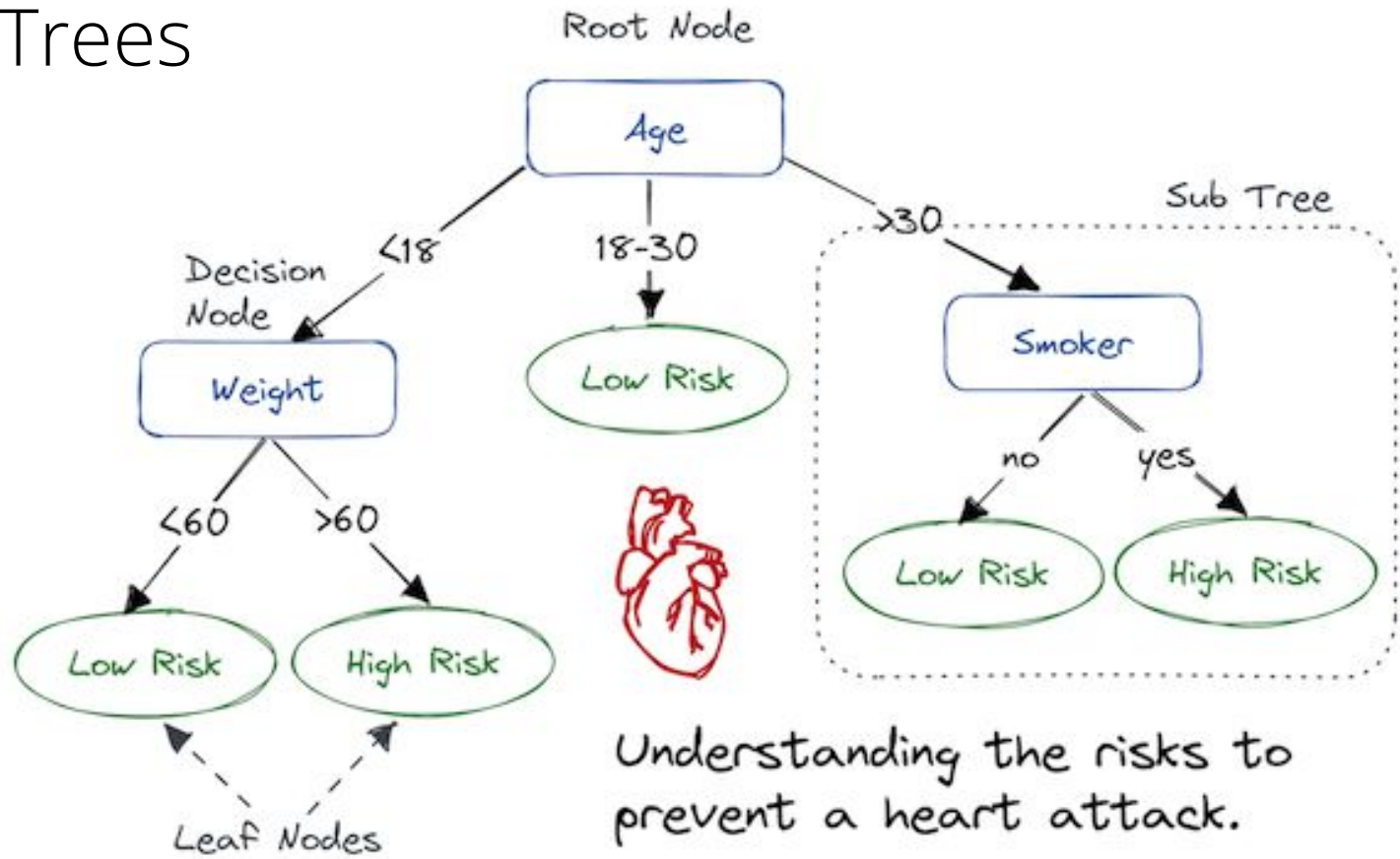


Jupyter Notebook

Part 2: Model Training Workflow

Decision Trees

Decision Trees



Decision Trees

A decision tree is a flowchart-like tree structure where an internal node represents a feature (or attribute), the branch represents a decision rule, and each leaf node represents the outcome.

The topmost node in a tree is known as the **root node**.

A **decision node** has two or more branches, each representing values for the attribute tested.

A **leaf node** represents a class label (in classification) or a continuous value (in regression).

A flowchart-like tree structure where an internal node represents a feature (or attribute), the branch represents a decision rule, and each leaf node represents the outcome.

Root Node

Decision Nodes

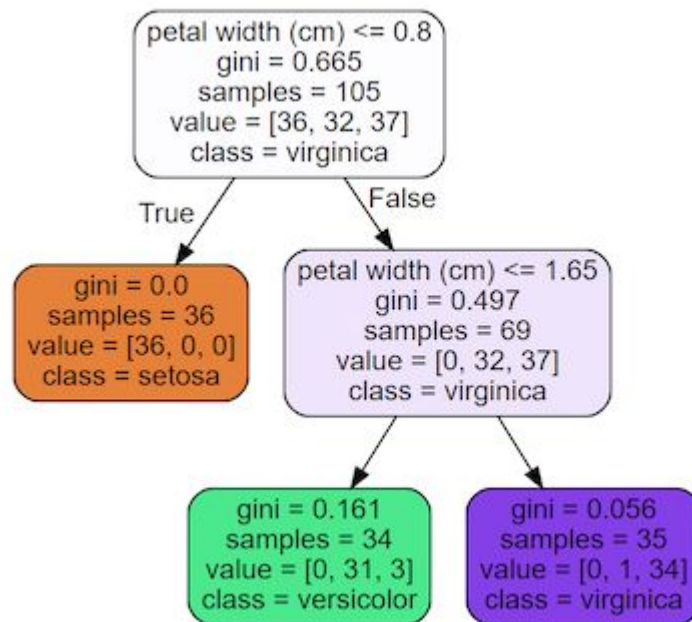
Has two or more branches, each representing values for the attribute tested.

Leaf Nodes

Represents a class label (in classification) or a continuous value (in regression).

Classification and Regression Trees (CART)

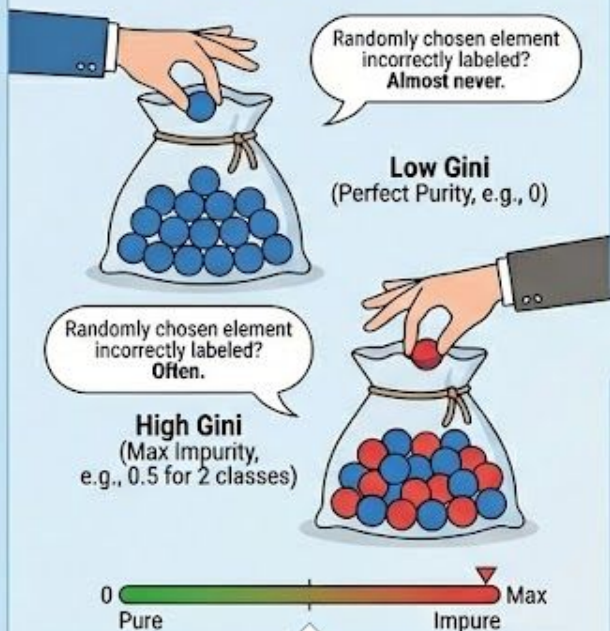
The most common algorithm used in sklearn is CART, which uses the **Gini impurity** or **Entropy** as a criterion to create splits.



BUILDING DECISION TREES:

Key Metrics for Splitting

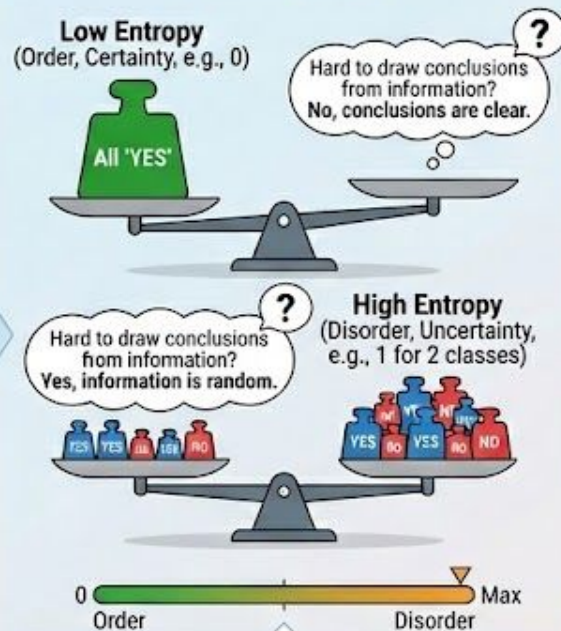
1. GINI IMPURITY



Measures how often a randomly chosen element from the set would be incorrectly labeled if labeled randomly according to the subset's distribution.

[cite: supervised_learning_3_lesson.ipynb]

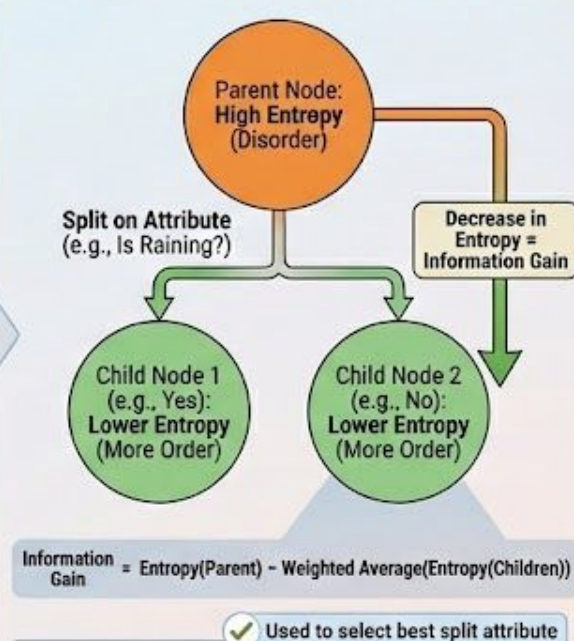
2. ENTROPY



Measures randomness or disorder. Higher entropy means greater uncertainty. Used to quantify impurity in decision tree nodes.

[cite: supervised_learning_3_lesson.ipynb]

3. INFORMATION GAIN



The decrease in entropy resulting from a split. Used to select the best attribute for splitting at each step.

[cite: supervised_learning_3_lesson.ipynb]

Pros vs Cons of Decision Trees

Advantages

- Simple to understand and interpret.
- Requires little data preprocessing.
- Can handle both numerical and categorical data.

Disadvantages

- Prone to overfitting, especially with a lot of features.
- Can be unstable because small variations in the data might result in a completely different tree.

Code-along



Jupyter Notebook

Part 3: Decision Trees

Ensemble Methods

Bagging, Boosting, Random Forest and
Gradient Boosting

Ensemble Learning

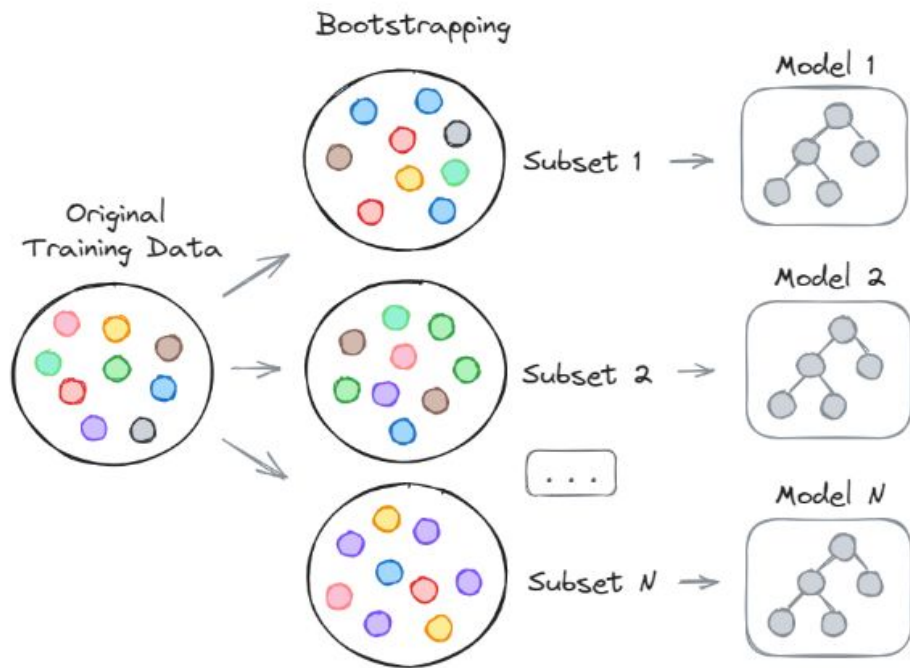
A powerful approach to improve the performance of machine learning models.

It combines multiple models to produce a single predictive model.

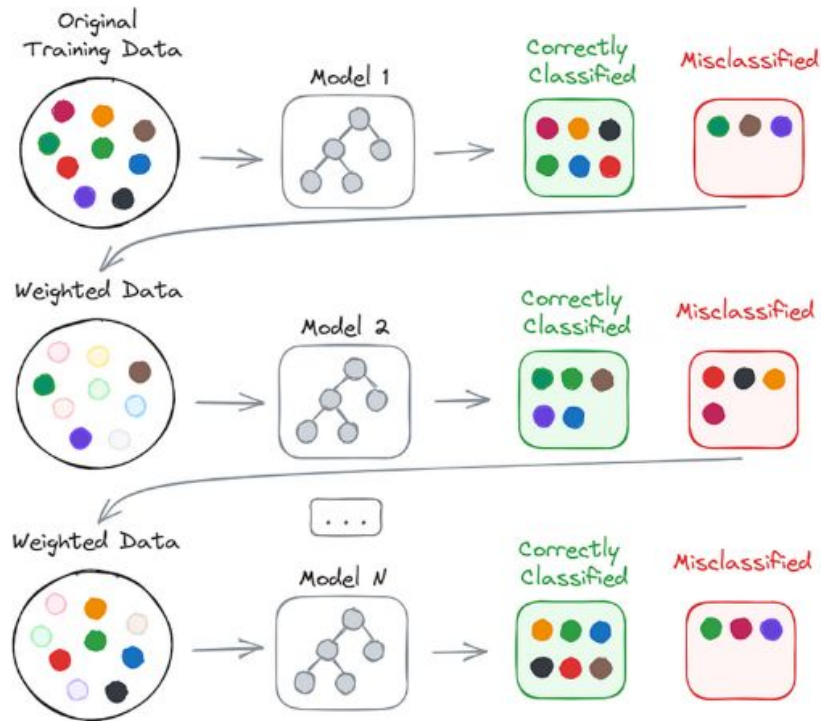
Two popular ensemble methods are bagging and boosting.



Bagging



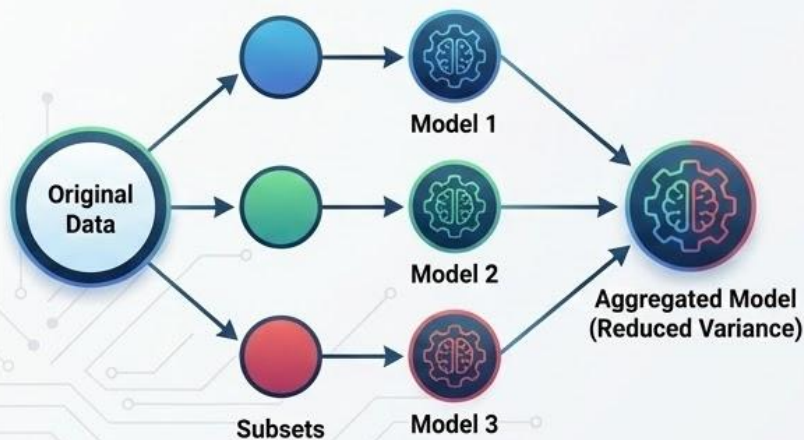
Boosting



Ensemble Methods

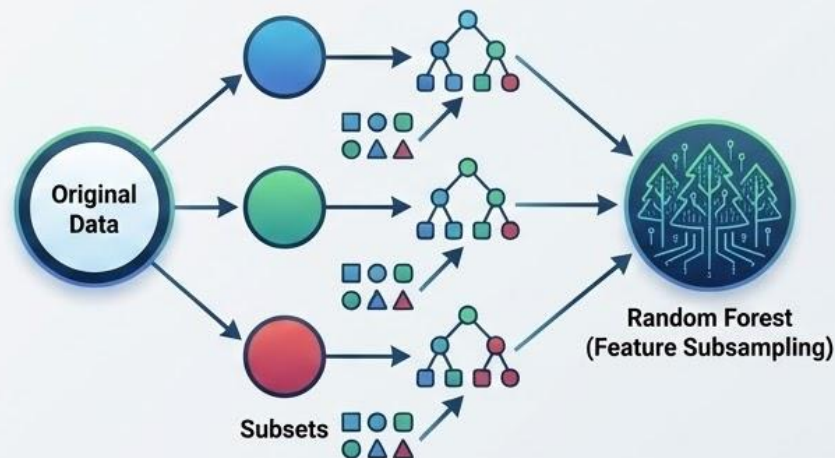
Bagging (*Bootstrap Aggregating*)

Aims to reduce the variance of predictive models by training multiple parallel models using different subsets of the training dataset.



Random Forest

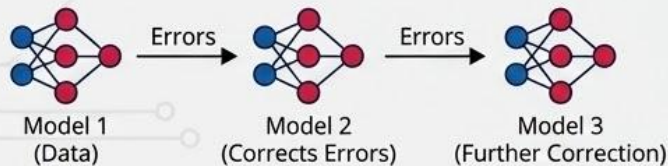
An extension of bagging applied to decision trees. In addition to training on different subsets of data, it also uses a subset of features for each tree.



Ensemble Methods

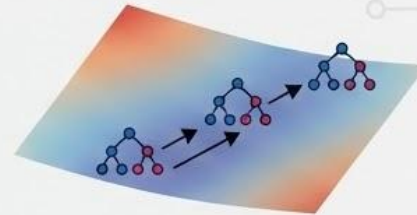
Boosting

Boosting creates a strong classifier from a number of weak classifiers. This is achieved by building a model from the training data, then creating a second model that tries to correct the errors from the first model.



Gradient Boosting

Gradient Boosting is a variant of boosting technique that uses gradient descent to minimize errors in sequential models. It builds one tree at a time, where each new tree helps to correct errors made by previously trained trees.



Advantages

- Often provides predictive accuracy that cannot be trumped.
- Lots of flexibility - can optimize on different loss functions and provides several hyperparameter tuning options that make the function fit very flexible.

LightGBM

What is LightGBM



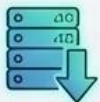
- LightGBM (**Light Gradient Boosting Machine**) is a gradient boosting framework developed by Microsoft that uses tree-based learning algorithms and is designed for distributed and efficient training, particularly on large datasets.
- LightGBM is known for its speed and performance, as well as its ability to handle large-sized data and support GPU learning.
- Currently one of the best Supervised Learning algorithms in terms of performance and speed for tabular (structural or dataframe-like) datasets.

Pros vs Cons

Advantages



Faster training and higher efficiency.



Lower memory usage.



Better accuracy with support for parallel and GPU learning.



Capable of handling large-scale data.

Disadvantages



May overfit on small datasets.



Requires careful tuning of hyperparameters.

Code-along



Jupyter Notebook

Part 4: Bagging and Boosting, LightGBM